Lecture 24: MIPS Memory Access Instructions (`lw`, `sw`, `lb`, `sb`, `lh`, `sh` & Addressing Modes)

🎯 Learning Objectives

By the end of this lecture, students will be able to:

- Understand why MIPS is called a **Load/Store Architecture**.
- Explain the difference between memory and registers.
- Use `lw`, `sw`, `lb`, `sb`, `lh`, and `sh`.
- Understand **base + offset addressing**.
- Explain memory alignment.
- Load variables from memory into registers.
- Store register values back into memory.
- Write complete MIPS programs using memory access instructions.

---

1. Introduction

🎤 Teaching Script

First 30-Second SEO Phrase

Naturally mention:

> **"In today's Computer Organization and Assembly Language lecture, we'll understand why MIPS is a Load/Store Architecture and learn how to use the `lw`, `sw`, `lb`, `sb`, `lh`, and `sh` instructions to move data between memory and registers with practical examples in the MARS simulator."**

In the previous lecture, we learned how to perform arithmetic operations using instructions such as `add`, `sub`, `mul`, and `div`.

But here's an important question:

Where do those numbers actually come from?

The answer is **memory**.

In MIPS, arithmetic instructions work only with registers. They cannot operate directly on variables stored in memory.

So before we can add, subtract, or multiply values, we first have to **load** them into registers. After performing the operation, we usually **store** the result back into memory.

This design makes MIPS a **Load/Store Architecture**.

In today's lecture, we'll learn how to move data between memory and registers using instructions like `lw`, `sw`, `lb`, `sb`, `lh`, and `sh`.

Let's begin.

---

2. Why Can't the CPU Work Directly with Memory?

Registers are located **inside the CPU**, making them much faster than RAM.

Memory is larger but slower.

MIPS arithmetic instructions such as:

add

sub

mul

only accept **register operands**.

For example, this is **invalid**:

add number1, number2, number3

Instead, we must:

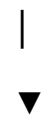
1. Load values from memory into registers.
2. Perform arithmetic using registers.
3. Store the result back into memory.

---

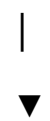
3. Load/Store Architecture

The data flow in MIPS is:

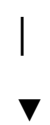
Memory



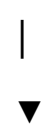
Load Instruction



Registers



ALU



Registers

Store Instruction

Memory

This approach keeps the instruction set simple and efficient.

---

4. Memory Layout in MIPS

Example:

.data

num1: .word 15

num2: .word 20

sum: .word 0

Memory might look like:

Address Value

10010000 15

10010004 20

10010008 0

Notice that each `.word` occupies **4 bytes**, so addresses increase by 4.

---

5. The `lw` (Load Word) Instruction

Purpose

Loads a **32-bit word** from memory into a register.

Syntax

lw destination, offset(base)

Example

lw $t0, num1

or equivalently using an address:

lw $t0, 0($s0)

Meaning:

Load the 32-bit value from memory into $t0.

---

6. Example Program Using `lw`

```
.data

number: .word 25

.text
.globl main

main:

    lw $t0, number

    li $v0, 10
    syscall
```

After execution:

$t0 = 25

---

7. The `sw` (Store Word) Instruction

Purpose

Stores a **32-bit register value** into memory.

Syntax

sw source, offset(base)

Example:

sw $t0, result

or

sw $t0, 0($s0)

Meaning:

Store the contents of $t0 into memory.

---

8. Example Using `lw` and `sw`

.data

num1: .word 10

result: .word 0

.text

.globl main

main:

```
lw $t0, num1

addi $t0, $t0, 15

sw $t0, result

li $v0, 10
syscall
```

Memory after execution:

result = 25

---

9. The `lb` (Load Byte) Instruction

Loads **1 byte (8 bits)** from memory into a register.

Syntax

```
lb destination, offset(base)
```

Example:

```
.data
```

grade: .byte 'A'

.text

lb $t0, grade

The ASCII value of `'A'` is loaded into `$t0`.

---

10. The `sb` (Store Byte) Instruction

Stores the **lowest 8 bits** of a register into memory.

Example:

sb $t0, grade

Useful for:

- Characters
- Small integer values

---

11. The `lh` (Load Halfword) Instruction

Loads a **16-bit value** into a register.

Example:

.data

marks: .half 95

.text

lh $t0, marks

---

12. The sh (Store Halfword) Instruction

Stores the **lowest 16 bits** of a register into memory.

Example:

sh $t0, marks

---

13. Addressing Mode in MIPS

MIPS primarily uses **Base + Offset Addressing**.

Syntax:

offset(base)

Example:

lw $t0, 8($s0)

Meaning:

Effective Address = Contents of $s0 + 8

If:

$s0 = 10010000

Then:

Address = 10010008

---

14. Visualizing Base + Offset

Base Address = 10010000

Offset = 8

Effective Address

10010000 + 8

↓

10010008

This addressing mode is heavily used for arrays and stack operations.

---

15. Memory Alignment

A **word** occupies 4 bytes.

Word addresses should be multiples of 4:

✓ 10010000

✓ 10010004

✓ 10010008

✓ 10010012

Misaligned accesses may not be supported on real MIPS hardware and can lead to exceptions.

---

16. Complete Program – Add Two Numbers from Memory

.data

num1: .word 15

num2: .word 25

sum: .word 0

```
.text
.globl main

main:

    lw $t0, num1
    lw $t1, num2

    add $t2, $t0, $t1

    sw $t2, sum

    li $v0, 10
    syscall
```

Memory after execution:

sum = 40

---

17. Viewing Memory in MARS

Demonstrate:

1. Assemble the program.
2. Open the **Data Segment**.
3. Observe `num1`, `num2`, and `sum`.
4. Step through the program.
5. Watch register values change.
6. See the `sum` variable update after `sw`.

This visual demonstration helps students connect memory with registers.

---

18. Real-Life Analogy

Imagine a classroom:

- **Memory** = Bookshelf
- **Registers** = Student's desk
- **CPU** = Student

The student first takes a book from the shelf (**load**), works on the desk (**registers + ALU**), and then returns the finished work to the shelf (**store**).

---

19. Common Mistakes

Students often:

❌ Try to add memory locations directly.

Incorrect:

add num1, num2, sum

✔ Correct approach:

lw $t0, num1

lw $t1, num2

```
add $t2, $t0, $t1

sw $t2, sum
```

❌ Confuse `lw` with `la`.

- `lw` loads the **value** stored at an address.
- `la` loads the **address** itself.

Example:

```
lw $t0, num1   # $t0 = 15


la $t0, num1   # $t0 = address of num1
```

❌ Use `lw` with `.byte` data.

✔ Use `lb` for bytes.

🎤 Lecture Summary

Today, we learned how data moves between memory and registers in MIPS Assembly Language.

We explored why MIPS is called a Load/Store Architecture and studied the `lw`, `sw`, `lb`, `sb`, `lh`, and `sh` instructions. We also learned about base + offset addressing, memory alignment, and the important difference between `lw` and `la`. Finally, we wrote complete programs that loaded values from memory, performed arithmetic, and stored the results back.

In the next lecture, we'll learn **MIPS Branch and Comparison Instructions**, including `beq`, `bne`, `slt`, `slti`, `j`, `jal`, and `jr`. These instructions allow us to make decisions and control the flow of a program, just like `if`, `else`, and loops in high-level languages.

Thank you for watching, and I'll see you in the next lecture.
